Bioinformatics Quality Management System

Ben Rambo-Martin, PhD
Lead Bioinformatics Scientist
US CDC – Influenza Division

Disclaimer

The findings and conclusions in this presentation are those of the authors and do not necessarily represent the official position of the Centers for Disease Control and Prevention.
Use of trade names and commercial sources is for identification only and does not imply endorsement by the U.S. Department of Health and Human Services.
References to non-CDC sites on the Internet do not constitute or imply endorsement of these organizations or their programs by CDC or the U.S. Department of Health and Human Services. CDC is not responsible for the content of pages found at these sites.

Objectives

- Describe quality management system (QMS) components
  - Reproducibility
  - Version control
  - Documentation

- Apply QMS principles to NGS workflows
  - Pre-analytical
  - Analytical

- Common troubleshooting

Reproducibility

- Same input + same methods = same results
  - Core principle of computational (all) science

- Record software versions
  - Aligners, samtools, variant callers, workflow managers
- Track changes with version control
  - Git for scripts and pipeline development
- Document parameters and references
  - Primer schemes
  - Quality thresholds

- Identify test data to run through new versions and pipelines

- Use workflow managers
  - Snakemake, Nextflow for structured, repeatable runs
- Leverage containers
  - Docker or Singularity ensure consistent environments

Reproducibility

- Same input + same methods = same results
  - Core principle of computational (all) science

- Record software versions
  - Aligners, samtools, variant callers, workflow managers
- Track changes with version control
  - Git for scripts and pipeline development
- Document parameters and references
  - Primer schemes
  - Quality thresholds

- Identify test data to run through new versions and pipelines

- Use workflow managers
  - Snakemake, Nextflow for structured, repeatable runs
- Leverage containers
  - Docker or Singularity ensure consistent environments

Reproducibility

- Same input + same methods = same results
  - Core principle of computational (all) science

- Record software versions
  - Aligners, samtools, variant callers, workflow managers
- Track changes with version control
  - Git for scripts and pipeline development
- Document parameters and references
  - Primer schemes
  - Quality thresholds

- Identify test data to run through new versions and pipelines

- Use workflow managers
  - Snakemake, Nextflow for structured, repeatable runs
- Leverage containers
  - Docker or Singularity ensure consistent environments

Reproducibility

- Same input + same methods = same results
  - Core principle of computational (all) science

- Record software versions
  - Aligners, samtools, variant callers, workflow managers
- Track changes with version control
  - Git for scripts and pipeline development
- Document parameters and references
  - Primer schemes
  - Quality thresholds

- Identify test data to run through new versions and pipelines

- Use workflow managers
  - Snakemake, Nextflow for structured, repeatable runs
- Leverage containers
  - Docker or Singularity ensure consistent environments

Documentation

- Explains how tools and pipelines work
  - Inputs, outputs, parameters, and assumptions
- Enables reproducibility
  - Others (and future you) can rerun the same analysis
- Reduces errors and misuse
  - Clear defaults and examples prevent incorrect runs
- Essential for collaboration
  - Shared understanding across labs, teams, and institutions

Documentation

- Explains how tools and pipelines work
  - Inputs, outputs, parameters, and assumptions
- Enables reproducibility
  - Others (and future you) can rerun the same analysis
- Reduces errors and misuse
  - Clear defaults and examples prevent incorrect runs
- Essential for collaboration
  - Shared understanding across labs, teams, and institutions

Documentation

- Explains how tools and pipelines work
  - Inputs, outputs, parameters, and assumptions
- Enables reproducibility
  - Others (and future you) can rerun the same analysis
- Reduces errors and misuse
  - Clear defaults and examples prevent incorrect runs
- Essential for collaboration
  - Shared understanding across labs, teams, and institutions

Documentation

- Explains how tools and pipelines work
  - Inputs, outputs, parameters, and assumptions
- Enables reproducibility
  - Others (and future you) can rerun the same analysis
- Reduces errors and misuse
  - Clear defaults and examples prevent incorrect runs
- Essential for collaboration
  - Shared understanding across labs, teams, and institutions

Best Practices in writing documentation

- README.md files
  - Overview, requirements, and basic usage
- Usage examples & command snippets
  - Copy-pasteable starting points
- Parameter descriptions
  - What flags do, expected values, and defaults
- Workflow diagrams
  - Visualize steps, inputs, and outputs

Best Practices in writing documentation

- README.md files
  - Overview, requirements, and basic usage
- Usage examples & command snippets
  - Copy-pasteable starting points
- Parameter descriptions
  - What flags do, expected values, and defaults
- Workflow diagrams
  - Visualize steps, inputs, and outputs

Best Practices in writing documentation

- README.md files
  - Overview, requirements, and basic usage
- Usage examples & command snippets
  - Copy-pasteable starting points
- Parameter descriptions
  - What flags do, expected values, and defaults
- Workflow diagrams
  - Visualize steps, inputs, and outputs

Best Practices in writing documentation

- README.md files
  - Overview, requirements, and basic usage
- Usage examples & command snippets
  - Copy-pasteable starting points
- Parameter descriptions
  - What flags do, expected values, and defaults
- Workflow diagrams
  - Visualize steps, inputs, and outputs

Best Practices in writing documentation

- Document data formats
    - FASTQ, FASTA, BAM, reference versions
- Record software versions
    - Tools, containers, workflow managers
- Include example datasets
    - Small test cases to validate setup

Best Practices in using documentation

- Start with the README
  - Understand the purpose, inputs, outputs, and limitations
- Check versions and dates
  - Ensure the documentation matches the tool or pipeline version you're using
- Follow the example first
  - Run the provided test or example before real data
- Read parameter descriptions carefully
  - Defaults may not match your experiment
- Verify input formats
  - FASTQ, FASTA, BAM, sample sheets, reference builds
- Look for expected outputs
  - Confirm files and directories match the documented results

Git

- Git is a version control system
  - Tracks changes to files over time
- Designed for collaboration
  - Multiple people can work on the same codebase
  - Think of it like a shared document (Microsoft office, Google docs)
- Keeps a history of your work
  - See what changed, when, why, and by whom
- Common in bioinformatics
  - Pipelines, scripts, documentation, and workflows

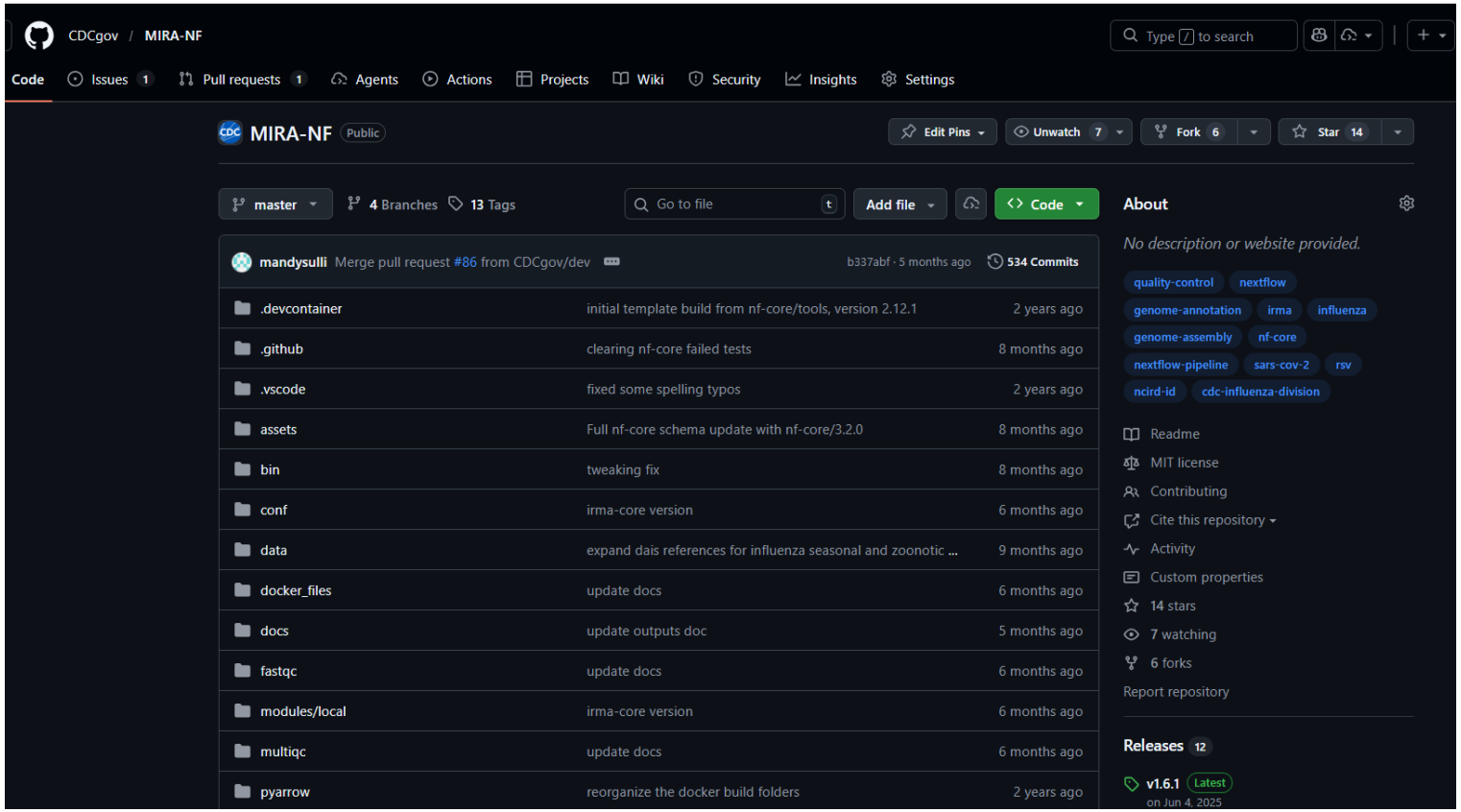
- Gitlab and Github most used

GitLab

GitHub

Core Git concepts

- Repository (repo)
  - A project tracked by Git
- Commit
  - A saved snapshot of changes with a message
- Branch
  - A parallel version of the code for development or testing
- Remote repository
  - Shared copy on platforms like GitHub or GitLab



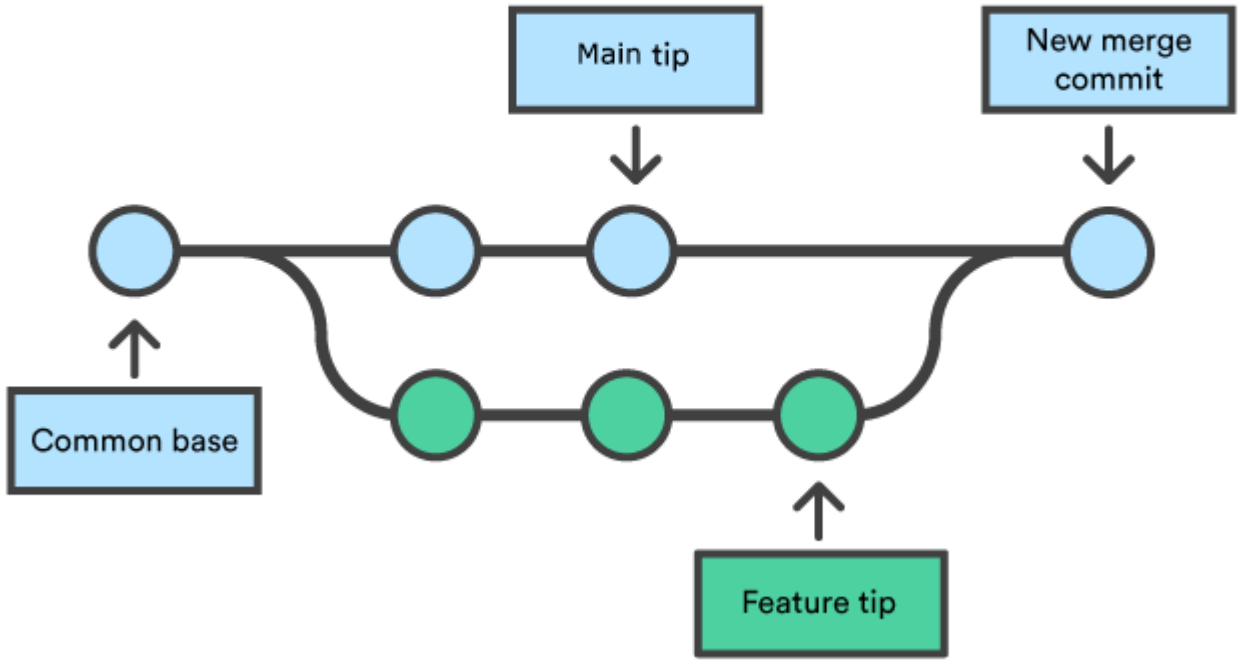

```
qgx6@rosalind02:MIRA-NF$ git status
On branch dev
Your branch is up to date with 'origin/dev'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .MIRA_nextflow.sh

nothing added to commit but untracked files present (use "git add" to track)
```

Git

- Common Git Commands (User Level)
- Create or get a repository
  - git init — start a repo
  - git clone <repo> — copy an existing repo
- Track and save changes
  - git status — see changes
  - git add <file> — stage changes
  - git commit -m "message" — save snapshot
- Sync with others
  - git pull — get updates
  - git push — share your changes



Common problems in Influenza bioinformatics

Ben Rambo-Martin, PhD
Lead Bioinformatics Scientist
US CDC – Influenza Division

Common issues

- Failed Mira QC
  - Low-coverage / Incomplete segment coverage
    - Not enough reads in your library → Back to the lab
      - Ct <= 28?
      - Gel image resolves segment bands cleanly?
      - Proper sample number on your flow cell?
    - MIRA <= v2.0.0 ??
      - Reads are being subsampled

Common issues

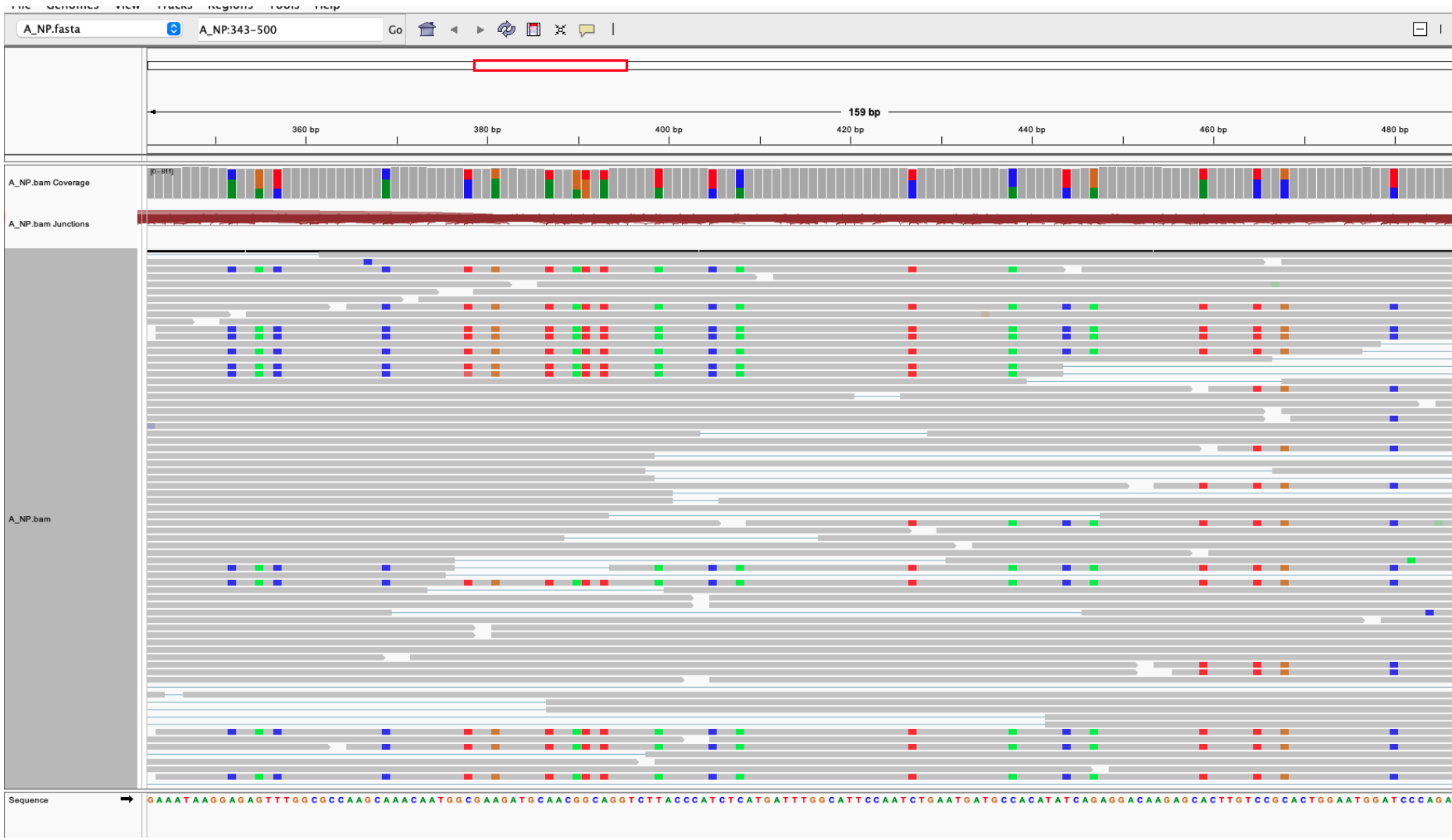
- Failed Mira QC
  - Minor variant count >10
    - In standard *clinical* samples, we have never observed >6 minor alleles (>=5% frequency) with the majority having 1-3, per segment
    - Cell cultures regularly have high genetic variability which can result in high counts
    - Could be a co-infection. Unlucky person got both H1N1 and H3N2 infection *at the same time*
    - **Most likely contamination!**

sample_id	total_reads	pass_qc	reads_mapped	reference	percent_reference_coverage	median_coverage	count_minor_snv_at_or_over_5_pct	pass_fail_reason
95f48e8a	95828	95828	10388	A_HA_H1	99.82	735	2	Pass
95f48e8a	95828	95828	5818	A_MP	100	686	148	Count of minor variants at or over 5% > 10
95f48e8a	95828	95828	9528	A_NA_N1	99.79	814	3	Pass
95f48e8a	95828	95828	9146	A_NP	100	722	274	Count of minor variants at or over 5% > 10
95f48e8a	95828	95828	5704	A_NS	97.1	777	147	Count of minor variants at or over 5% > 10
95f48e8a	95828	95828	11790	A_PA	100	607	361	Count of minor variants at or over 5% > 10
95f48e8a	95828	95828	13230	A_PB1	100	687	246	Count of minor variants at or over 5% > 10
95f48e8a	95828	95828	12175	A_PB2	100	590	391	Count of minor variants at or over 5% > 10
c282a097	16170	16170	1992	A_HA_H3	99.82	158	35	Count of minor variants at or over 5% > 10
c282a097	16170	16170	1382	A_MP	100	187	12	Count of minor variants at or over 5% > 10
c282a097	16170	16170	1852	A_NA_N2	100	178	25	Count of minor variants at or over 5% > 10
c282a097	16170	16170	1720	A_NP	100	157	18	Count of minor variants at or over 5% > 10
c282a097	16170	16170	1336	A_NS	97.1	206	10	Pass
c282a097	16170	16170	2516	A_PA	100	148	22	Count of minor variants at or over 5% > 10
c282a097	16170	16170	2712	A_PB1	100	161	23	Count of minor variants at or over 5% > 10
c282a097	16170	16170	2660	A_PB2	100	160	28	Count of minor variants at or over 5% > 10

Two populations are present.

One matching reference (gray) one sharing all the colored SNVs

2+ mutations on one molecule are “phased” or “linked”

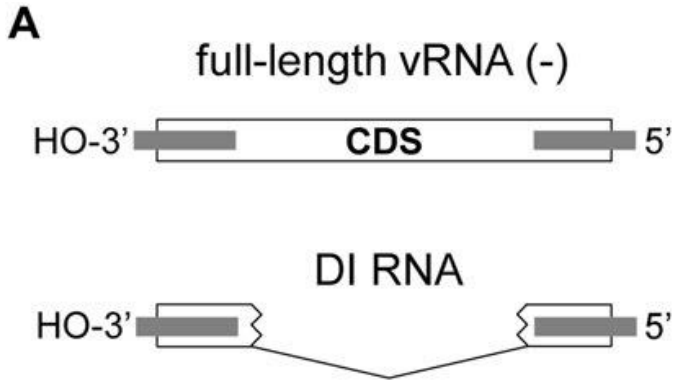


Common issues

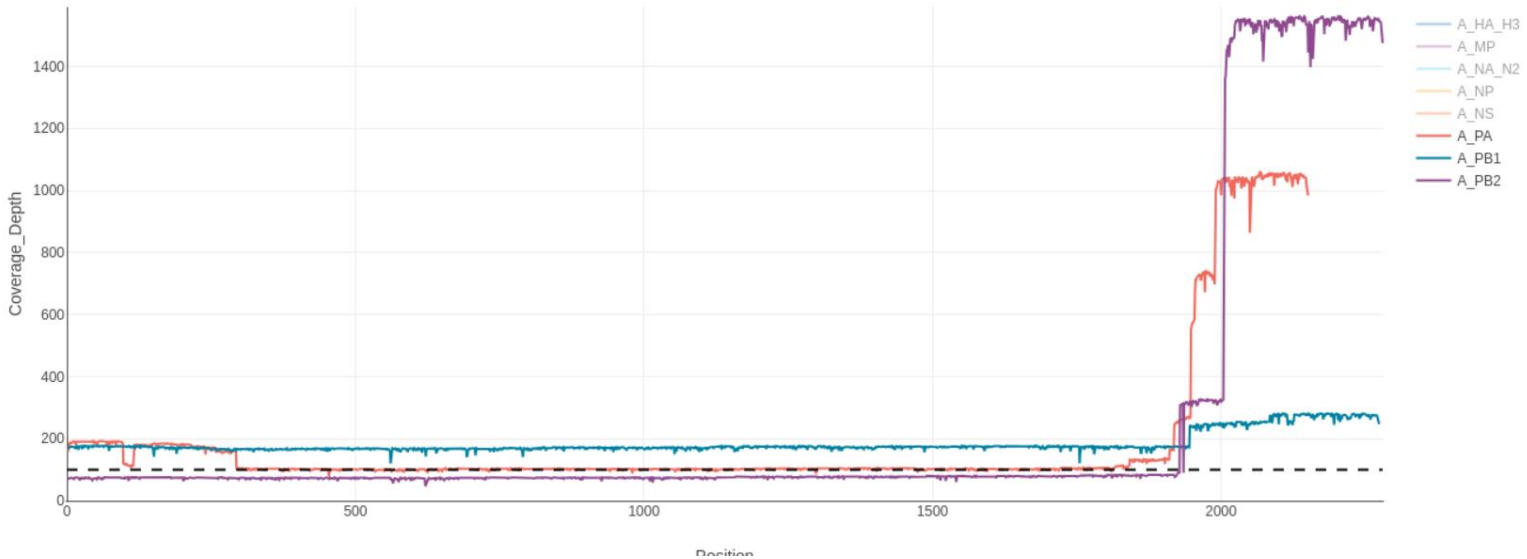
- Failed Mira QC
  - Premature stop-codon?
    - ONT homopolymer issue may require manual correction
    - DI particle induced alignment

DI Particles

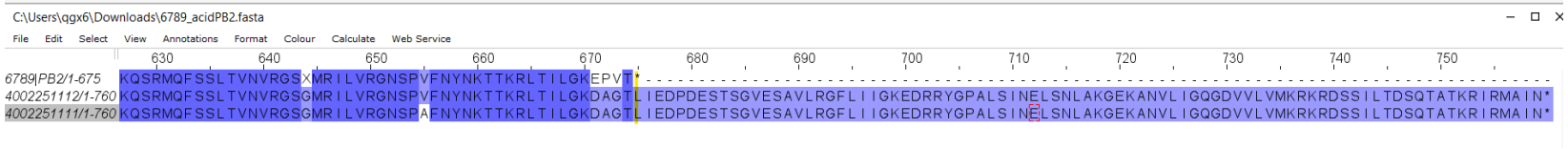
- DI Particles can interfere with NGS
- Common in polymerase segments
- Coverage shape can spike at end or “bat ears”
- Can create erroneous indel mutations at coverage dropoff points



https://analyticalsciencejournals.onlinelibrary.wiley.com/doi/10.1002/biot.201400429

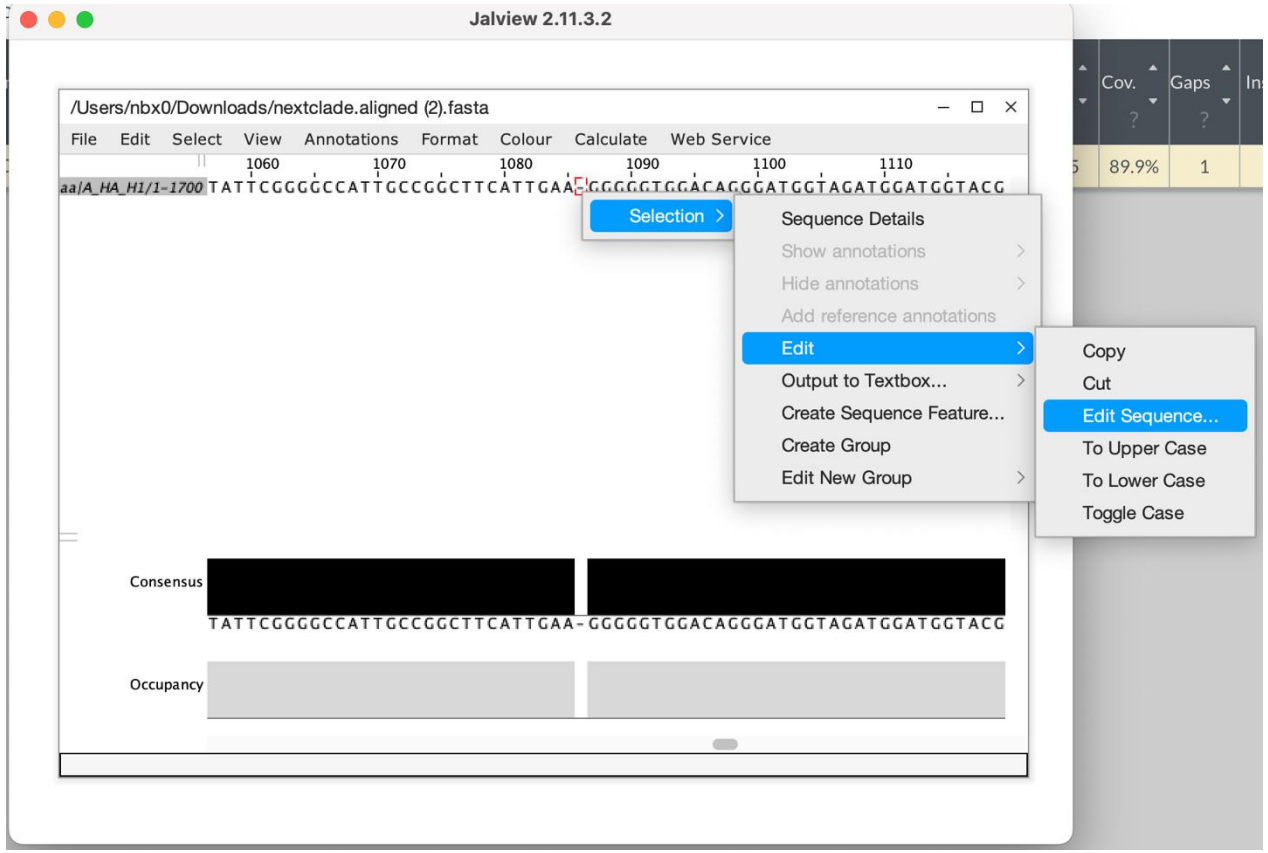


DI Particles



Frameshifts

- Prevalent in homopolymer regions of Oxford Nanopore Sequencing
- DAIS-Ribosome is frameshift-tolerant
  - Shows as (~) mutation
- Convert back to nucleotide space and add or remove base to fix



Mira demands high quality results

- Together we can stop the “Garbage In” side of the data analysis mantra: “Garbage In, Garbage Out”
- Built-in thresholds are for standardizing QC
- Amended consensus gives us extra information from a single sequence.
  - A mixed site may be under active or balancing selection in the host
- Consider each sample as a whole: if HA and NA pass but all other segments fail, consider why.
- High priority samples can be useful even when QC thresholds are not met

Mira demands high quality results

- Together we can stop the “Garbage In” side of the data analysis mantra: “Garbage In, Garbage Out”
- Built-in thresholds are for standardizing QC
- Amended consensus gives us extra information from a single sequence.
  - A mixed site may be under active or balancing selection in the host
- Consider each sample as a whole: if HA and NA pass but all other segments fail, consider why.
- High priority samples can be useful even when QC thresholds are not met

Mira demands high quality results

- Together we can stop the “Garbage In” side of the data analysis mantra: “Garbage In, Garbage Out”
- Built-in thresholds are for standardizing QC
- Amended consensus gives us extra information from a single sequence.
  - A mixed site may be under active or balancing selection in the host
- Consider each sample as a whole: if HA and NA pass but all other segments fail, consider why.
- High priority samples can be useful even when QC thresholds are not met

Mira demands high quality results

- Together we can stop the “Garbage In” side of the data analysis mantra: “Garbage In, Garbage Out”
- Built-in thresholds are for standardizing QC
- Amended consensus gives us extra information from a single sequence.
  - A mixed site may be under active or balancing selection in the host
- Consider each sample as a whole: if HA and NA pass but all other segments fail, consider why.
- High priority samples can be useful even when QC thresholds are not met

Mira demands high quality results

- Together we can stop the “Garbage In” side of the data analysis mantra: “Garbage In, Garbage Out”
- Built-in thresholds are for standardizing QC
- Amended consensus gives us extra information from a single sequence.
  - A mixed site may be under active or balancing selection in the host
- Consider each sample as a whole: if HA and NA pass but all other segments fail, consider why.
- High priority samples can be useful even when QC thresholds are not met